UMHACKATHON 2026 · DOMAIN 1 · AI SYSTEMS & AGENTIC WORKFLOW AUTOMATION

Technical Architecture.

Stack, ILMU/GLM integration, the workflow engine, and the trade-offs we made to ship the MVP.

TEAM · OPUS MAGNUM

Oybek Odilov
Al-Bazeli Gameel Abduljalil
Lee Yih Shen
Samandar Erkinjonov
Abdugaffor Odilov

ONBOARD

POWERED BY Z.AI GLM · REGULATED ONBOARDING ENGINE

TAD · TECHNICAL ARCHITECTURE APRIL 2026 SUBMISSION RELEASE

1. Stack at a Glance

LAYER	CHOICE	VERSION	WHY
Reasoning engine	ILMU `ilmu-glm-5.1` via OpenAI SDK	`openai>=2.32`	Mandatory for the hackathon; full OpenAI surface (tools, JSON mode, streaming)
Backend language	Python	3.12	First-class LangGraph + Pydantic + OpenAI SDK ecosystem
Web framework	FastAPI + Pydantic v2	0.136 / 2.13	Typed request/response, SSE via `sse-starlette`, zero boilerplate
Orchestration	LangGraph	1.1	Native stateful graphs, checkpointer, conditional edges, human-in-loop
Checkpointer	`langgraph-checkpoint-sqlite` / `-postgres`	3.0 / 3.0	Dev + prod with the same interface
Database	Postgres (prod) / SQLite (dev)	16 / bundled	JSONB on both (TEXT on SQLite); SQLAlchemy 2.0 maps abstractly
PDF extraction	`pypdf`	6.10	Pure Python, text-layer first
OCR fallback	`pdf2image` + `pytesseract` (`ocr` extra)	0.3	Kicks in when PDFs lack a text layer
Frontend	Next.js App Router + TypeScript	16.2 / 5.9	Routing, server-rendered directory, static landing
Styling	Tailwind CSS	4	Dark token system; zero-config with App Router
Workflow canvas	`@xyflow/react`	12.10	Custom step nodes per primitive with live status
3D	react-three-fiber + drei + three	9.6 / 10.7 / 0.184	Real GLB apartment models (CC-BY from Poly Pizza)

LAYER	CHOICE	VERSION	WHY
Realtime	Server-Sent Events	native	Simpler than WS for one-way token streams
Auth	Mocked session (seeded users)	—	Out of scope; flagged in PRD

2. GLM / ILMU Integration

2.1 Endpoint and SDK

- **Base URL** `https://api.ilmu.ai/v1`
- **Auth** `Authorization: Bearer $GLM_API_KEY`
- **SDK** `openai.OpenAI(api_key=..., base_url=...)` — ILMU is fully OpenAI-compatible.

Single point of contact: `backend/app/glm/client.py` exposes `GLMClient` with three public surfaces:

- `complete(...)` — free-form chat, optional streaming.
- `complete_json(...)` — JSON-mode with schema-hint system prompt.
- `call_tool(tool_spec=...)` — forced function call, returns parsed arguments.

All three:

- retry `RateLimitError` / 5xx with exponential backoff + jitter,
- emit a `Decision` to an optional sink (the primitive-level decision log),
- default to `reasoning_effort="low"` on structured-output paths (see §2.3).

2.2 Capabilities verified empirically

Smoke tests in `backend/scripts/glm_smoke.py` resolve `implementation.md §19`:

CAPABILITY	RESULT	EVIDENCE
Plain chat completion	✓	`smoke_plain` returns expected word
JSON mode (`response_format={"type":"json_object"}`)	✓	`smoke_json` matches schema
Tool use / function calling	✓	`smoke_tool_use` returns correct structured args
Streaming (SSE)	✓	4+ delta chunks observed
`reasoning_effort="low/medium/high"`	✓	Standard param; unlocks thinking budget
Vision / image inputs	✗	Model replies "I cannot see any image" for any PNG payload; OCR fallback is the baseline

All seven product tools were validated against real model output — see `implementation.md §19` for the resolved findings.

2.3 Reasoning-effort discipline

`ilmu-glm-5.1` is a **thinking model**: part of the token budget goes to internal reasoning. Two practical consequences:

- **Small `max_tokens` ceilings empty `content`.** Under 256 tokens is unsafe for free-form output. Our client logs a warning and retries, but the production-correct choice is to budget ≥ 512 tokens on visible responses.
- **Structured outputs run well on `reasoning_effort="low"`.** `call_tool` and `complete_json` default to `low` — extraction and verification do not benefit from deep reasoning and `low` is materially faster.

Free-form assistant streams (visitor chat, template tutor) omit `reasoning_effort` so the model's default applies; the system prompt governs tone and grounding discipline instead.

2.4 Audit trail: `Decision`

Every GLM call produces a `Decision` dataclass:

```
{
  "call_id": "...",
  "tool": "verify_against_criteria",
  "request": {"model": ..., "messages": [...redacted...], "tools": [...]},
  "response": {"content": ..., "tool_calls": [...], "finish_reason": ..., "usage":
  "duration_ms": 842,
  "ts": 1714032384.1,
  "step_id": "ownership"   # tagged by the primitive
}
```

Primitives pass a local sink to `GLMClient(sink=...)` so the decisions land tagged with the owning step_id. The runner merges these into `step_runs.decision_log` on every tick. Long content fields are elided to keep the log scannable.

3. Workflow Engine: Template → StateGraph

3.1 Primitive vocabulary

Six primitives — the full vocabulary for all templates:

PRIMITIVE	GLM TOOLS USED
`collect_form`	optional `explain_field`
`upload_compliance`	`extract_document`, `verify_against_criteria`
`upload_content`	`extract_document`
`cross_check`	`cross_check_documents`
`human_review`	`summarize_for_review`
`publish`	— (terminal)

3.2 Compiler

`compile_template(steps)`:

1. Constructs a `StateGraph(RunState)`.
2. For each step, looks up the factory in `REGISTRY` (populated by decorated primitive modules) and adds a node with the step's id.

3. For each step, adds a conditional edge keyed by `_route_after_step`: route to `END` when `awaiting_user` is true, otherwise to the next step (or `END` for the last).
4. Sets the entry point and compiles with the passed-in checkpointer.

Conditional routing per primitive (e.g. upload_compliance re-upload loop) is expressed by the primitive's own return values — the compiler is agnostic.

3.3 RunState reducers

`RunState` is a `TypedDict` with reducers on append-only fields:

- `step_outputs` — shallow dict merge (later tick wins per key).
- `uploaded_docs`, `verification_results`, `decision_log` — list concatenation.
- `profile_draft`, `pending_input` — shallow dict merge.

The reducers guarantee that each node return is a *patch*, not a full state replacement — primitives never accidentally erase prior work.

3.4 Human-in-the-loop handshake

LangGraph supports `interrupt()` natively, but we chose a simpler "paused-by-flag" model:

- Primitive returns `awaiting_user=True, awaiting_step_id=<step>` when it needs input.
- `_route_after_step` routes to `END` on that flag.
- API endpoint submitting the step merges the payload into `pending_input[step_id]` and re-invokes the graph.
- Primitive sees `pending_input[step_id]`, consumes it, and advances.

Advantages: transparent state (the pause is a state flag, not an interrupt token), trivial to test, trivial to resume. Trade-off: one extra `if pending_input[step_id]` check per primitive; worth the clarity.

4. Frontend Architecture

4.1 Next.js 16 App Router

- Server components for data-heavy read pages (public directory).
- Client components for interactive surfaces (workflow canvas, step panel, assistant chat).
- Dynamic routes with `params: Promise<{...}>` unwrapped via `use(params)` (Next 16 convention).

4.2 API client

`frontend/lib/api.ts` is a small typed fetch wrapper. Types mirror backend Pydantic models by hand — changes to the API are caught at compile time.

4.3 SSE helper

`frontend/lib/sse.ts` parses `text/event-stream` from POST responses using a manual `fetch` + `ReadableStream` reader. Native `EventSource` is GET-only and not usable for chat-POST flows.

4.4 React Flow canvas

Custom `StepNode` type with a per-primitive icon, live status badge (`pending / running / awaiting_user / passed / failed`), and a pulse-ring animation on states that need attention. Click routes to the step panel.

4.5 3D scene

`react-three-fiber` canvas loading a CC-BY GLB apartment model from `public/models/`. Each building gets a deterministic model via a hash of its id, so repeated visits show the same building but the directory has visual variety. Models are normalised via a bounding-box scale-to-fit so different source scales render consistently. Scene-config stats (floors / units / footprint) are overlaid through `drei`'s `<Html>`.

5. Persistence & Storage

5.1 Database

SQLAlchemy 2.0 with `DeclarativeBase`. One session per request (FastAPI `Depends`). JSON columns use `sqlalchemy.JSON` which maps to JSONB on Postgres, TEXT on SQLite — app code uses the same types regardless.

Schema bootstrap via `Base.metadata.create_all()` — alembic was deliberately deferred for hackathon velocity (PRD §4.2).

5.2 Object store

`STORAGE_BACKEND=local` writes uploaded files under `var/uploads/<building_id>/<step_id>/<doc_id>__<filename>`. Production path is S3 with the same key layout; only the backend picker needs to change.

5.3 LangGraph checkpointer

- **Dev:** SQLite file at `var/langgraph.sqlite` (`SqliteSaver`).
- **Prod:** Postgres at the same DATABASE_URL (`PostgresSaver`).

`runner.py` picks the right saver by reading the URL scheme; callers never touch the checkpointer directly.

6. Key Trade-offs

DECISION	ALTERNATIVE CONSIDERED	RATIONALE
LangGraph over custom state machine	Hand-rolled Python generator	Built-in checkpointer, human-in-loop interrupt semantics, conditional edges, ecosystem
Template as JSON in DB	YAML files in repo	Admin UX: the admin *is* the author at runtime; file-system templates would require redeploy per city
Two doc classes (compliance vs content)	One class with a role tag	Clearer UI contract; different verification vs extraction flows; never shown publicly vs shown publicly is a user-facing invariant
SSE over WebSocket	WebSocket	One-way streams suffice; SSE is trivially resumable and CORS-friendly
`create_all` over Alembic	Alembic	Schema is tiny; any change in the hackathon window is a full reset. Alembic is the production upgrade path
Real GLB apartment models	Extruded-box placeholders	Visual payoff matters for pitch judging; CC-BY models from Poly Pizza cost nothing and look like real buildings
Fresh-compile per tick	Compile-once cache	The cache held stale checkpointer references across requests. Compile is cheap (microseconds); correctness first
OpenAI SDK over raw HTTP	`httpx` + hand-rolled retry	Retry semantics, streaming iterator, structured-output typing all already in the SDK

7. Dev Experience

```
# one-time
just install           # venv + pip + pnpm install
cp .env.example .env   # then set GLM_API_KEY

# daily
just db-up             # Postgres via docker compose (or skip for SQLite)
just migrate           # create_all schema bootstrap
just seed              # admin, owner, Shah Alam, template, Menara Demo
just backend-dev       # uvicorn on :8000
just frontend-dev      # next dev on :3000

# quality
just test              # pytest (unit + integration; live-GLM marked skipif)
just smoke-glm         # resolve / re-resolve open questions in §19
```

A Makefile mirrors the justfile for environments without `just`.

8. Known Limitations

- No vision path on `ilmu-glm-5.1` (empirically confirmed). Images degrade to OCR text.
- Single-tenant demo: mocked auth, no per-user isolation beyond the seeded ids.
- SSE endpoint uses a 500ms poll of the DB. Fine for one concurrent demo run; a pub/sub bus (Redis) is the upgrade path.
- No embeddings / RAG. Visitor assistant grounds on the structured profile + raw content-doc text pulled in-context (GLM has 200k context; single-building content is well within budget).

9. Security Notes (within the MVP scope)

- `.env` with `GLM_API_KEY` is gitignored. `.env.example` ships the variable names only.
- FastAPI CORS allows only `http://localhost:3000`.
- Uploaded files are written under `var/uploads/…`; filename is sanitised to a basename before writing.
- No SQL string concatenation — all queries are SQLAlchemy ORM.
- Decision logs redact message bodies over 4k chars and elide `image_url` content — keeps the log readable and doesn't cache whole documents.

See `sad.md` for data flows and module boundaries; `implementation.md` for section-by-section engineering plan; `backend/scripts/glm_smoke_results.json` for the current capability matrix.